

Stock Market Analysis Project Using NumPy

Objectives

- Analyze stock closing prices using NumPy.
- Perform basic and statistical analysis, filtering, broadcasting, and memory optimization, Reshaping, Flattening.

```
import numpy as np
# stock Closing prices for 10 days of single company
close_price = np.array([150, 152, 149, 153, 155, 157, 156, 160, 162, 161])
```

Basic Analysis

```
print("Shape of Stock Closing price:", close_price.shape) # shape(row X columns )
print("Size of the Stock Close Prices :", close_price.size) #Total Elements in the a
print("Dimension of stock close price :", close_price.ndim)
print("datatype of close price of stock:", close_price.dtype)
print("Minimum Stock closing Price in !0 days :", "Rs.", np.min(close_price))
print("Maximum Stock closing price in 10 days : ", "Rs.", np.max(close_price))
print("Sum of closing price of 10 days :", "Rs.", np.sum(close_price))
print("For 2nd day closing price of stock: ", "Rs.", close_price[00]) # slicing
```

```
Shape of Stock Closing price: (10,)
Size of the Stock Close Prices : 10
Dimension of stock close price : 1
datatype of close price of stock: int32
Minimum Stock closing Price in !0 days : Rs. 149
Maximum Stock closing price in 10 days : Rs. 162
Sum of closing price of 10 days : Rs. 1555
For 2nd day closing price of stock: Rs. 150
```

Memory Before Optimzation

```
print("Total Memory (bytes):", close_price.nbytes, "Bytes ") # nbytes gives itemsize
print("Total Memory for each close price (bytes):", close_price.itemsize, "Bytes ")
```

```
Total Memory (bytes): 40 Bytes
Total Memory for each close price (bytes): 4 Bytes
```

Memory Optimization and Changing Data type

```
optimize=close_price.astype(np.float16) #changes int datatype into float and optimi
print("Original data Close_price Array:", close_price.dtype)
print("Optimized Datatype of close price Array:", optimize.dtype)
print("Optimized Memory bytes :", optimize.nbytes, "Bytes")
```

```
Original data Close_price Array: int32
Optimized Datatype of close price Array: float16
Optimized Memory bytes : 20 Bytes
```

Statistical Analysis for Stock Price

```

print("Average of 10 days Stocks of 10 days Closing Price :", "RS.", np.mean(close_p
print("Median stock of 10 days closing price: ", "RS.", np.median(close_price))
print("Standard deviation for 10 days stock closing price:", "RS.", np.std(close_pric
print("Variance having stock of 10 days closing price:", "RS.", np.var(close_price))

```

```

Average of 10 days Stocks of 10 days Closing Price : RS. 155.5
Median stock of 10 days closing price: RS. 155.5
Standard deviation for 10 days stock closing price: RS. 4.318564576337836
Variance having stock of 10 days closing price: RS. 18.65

```

Daily Returns After Closing The Stock

```

daily_returns = np.diff(close_price) / close_price[:-1] * 100 #finds difference bet
print("Everday Returns after closing the stocks :\n ", daily_returns)
print("Daily Returns (%):", np.round(daily_returns, 2)) # round gives rounded value

```

```

Everday Returns after closing the stocks :
[ 1.33333333 -1.97368421  2.68456376  1.30718954  1.29032258 -0.63694268
 2.56410256  1.25          -0.61728395]
Daily Returns (%): [ 1.33 -1.97  2.68  1.31  1.29 -0.64  2.56  1.25 -0.62]

```

Filtering the data

```

high_price=close_price[close_price>156]
print("stock peak values in 10 days : ", high_price, "Rs")

```

```

low_price=close_price[close_price<150]
print("Lowest stock closing price in 10 days : ", low_price, "Rs")

```

```

stock peak values in 10 days : [157 160 162 161] Rs
Lowest stock closing price in 10 days : [149] Rs

```

Conditional Filtering (If-Else Condition) for Alerts

```

today_price = close_price[-1] # indexing negative
if today_price > np.mean(close_price):
    print(" Increase The stock Price of today ")
else:
    print("Possible the Downfall and Decreases the stock closing Price ")

```

```

    Increase The stock Price of today

```

Reshape the Stock Close Price Array

```

Reshaped_arr=close_price.reshape(2, 5) # reshape the original array of 1D to multiD
print("reshaped array of stock close prices :\n", Reshaped_arr)

```

```

reshaped array of stock close prices :
[[150 152 149 153 155]
 [157 156 160 162 161]]

```

Flatten the Array

```

Flattened_arr=Reshaped_arr.flatten( ) # convert reshaped 2d array into 1 dimensiona
print("Flattened array of Stock Closing Prices :\n ", Flattened_arr)

```

```

Flattened array of Stock Closing Prices :
[150 152 149 153 155 157 156 160 162 161]

```

Broadcasting Grow the Stock over 10 days

```
# stock prices increases by rs.10 (bonus adjustments)
```

```
New_prices=close_price+ (10 )
```

```
print("All Stock Prices After 10 days :", "Rs.", np.round(New_prices,0))
```

```
All Stock Prices After 10 days : Rs. [160 162 159 163 165 167 166 170 172 171]
```

- Stocks prices grow in rs.10 in each close price.
- Helps in business of company to grow in market.

Final Project Summary

- Minimum closing price of stock Rs.149
- Maximum closing price in 10 days of stock Rs.162
- Average closing Price in 10 days of stock Rs. 155.50
- Median closing Price in 10 days of stock Rs.155
- standard deviation is Low.
- New prices after bonus (10 days) projected some closing price of Stock.

Stock Market Impact

- The stock grew overall from Rs.150 to Rs. 161 (+Rs.11).
- Average stock price is around Rs. 155.5.
- Currently trading above average then Stock Price Uptrend.
- Memory optimized version uses half the space (20 bytes).
- Broadcasting efficiently simulated a 5% price increase.

Conclusion

In numpy, I have done analysis on stock price data and findout daily return of stock and extracted Quick insights and using filtering we understood when stock closing price is on peak or low.